

Domain Class Diagram — draw.io layout

Hand-routed orthogonal layout (the same "bus routing" technique draw.io's own auto-layout uses): one shared inheritance trunk into **BaseEntity** at top, and each association/dependency runs its own horizontal lane below the class row. Dashed = dependency (**RegistrationInvite**'s polymorphic resolution). A «**proposed**» tag marks classes not yet in the codebase. Each relationship line has its own color purely to make it easy to trace individually where several run close together — this is separate from the box fill colors below, which group classes by area. Multiplicities (0..1, 1, 0..*) are shown at each end, per standard UML notation; dependencies carry none, since they aren't structural associations. Each entity now also shows its Lombok-generated constructors (+**ClassName**() , +**ClassName**(...)). The technique repeats three times below the entity row: enums are dependency targets reached by a lane from above; six «**service**» classes (methods and constructor read from the actual ***ServiceImpl** code) sit as dependency sources routed up into the entity/entities each one uses; and seven «**interface**» repositories (Spring Data JPA, read from the actual code) sit one tier deeper still, as the dependency targets each service's constructor injects. Each «**controller**» sits directly left of its one service with a short direct arrow (methods read from the actual ***Controller** code); three controllers also reach across to the proposed **RegistrationInviteService** for issuing/claiming an invite — both ends are coplanar, so those arrows hop through the same lane band above the row rather than crossing the repository row below. An editable **.drawio** file with this exact layout accompanies this PDF.

Base / audit Identity & access Academic profiles Academic catalog Enumeration Association (Enrollment)

